



RAPPORT DE TP 1

Introduction à OpenGL

Présenté par :

Fatma Zohra Chakroun
Abdellah Benali

Année Universitaire : 2025 – 2026

Table des matières

1	Introduction	2
2	Exercice 1 : Premiers pas et Interactivité	2
3	Exercice 2 : Perspective et Caméra	2
4	Exercice 3 : La Pile de Matrices	5
5	Exercice 4 : Navigateur Interactif	5
5.1	Logique de la vue et calcul de direction	6
5.2	Contrôles et Interactivité	6
6	Conclusion	7

1 Introduction

Ce TP présente les bases d'OpenGL : l'affichage d'objets 3D (la théière), la gestion de la caméra et l'utilisation de la pile de matrices pour organiser la scène.

2 Exercice 1 : Premiers pas et Interactivité

Affichage : Nous avons comparé le rendu filaire (*wire*) et plein (*solid*). Sans lumière, l'objet plein n'a pas de relief. Pour la sphère, plus on augmente les *slices* et *stacks*, plus elle est ronde.

Fenêtrage : Pour éviter de déformer la théière quand on redimensionne la fenêtre, nous avons forcé le *viewport* à rester un carré centré pour éviter la déformation de l'objet.

Clavier : La touche 'w' a été programmée pour basculer entre le mode plein et le mode filaire via la fonction `glPolygonMode`.



FIGURE 1 – rendu plein



FIGURE 2 – rendu avant réglage de viewport

3 Exercice 2 : Perspective et Caméra

Champ de vision (FOV) : Plus le FOV est grand, plus la théière semble petite car on voit plus large (effet grand angle). À l'inverse, un petit FOV fait un effet de zoom.

Plan de coupe (zNear) : Si on augmente trop la valeur de $zNear$, le plan de coupe traverse la théière. On voit d'abord l'intérieur, puis elle disparaît complètement.

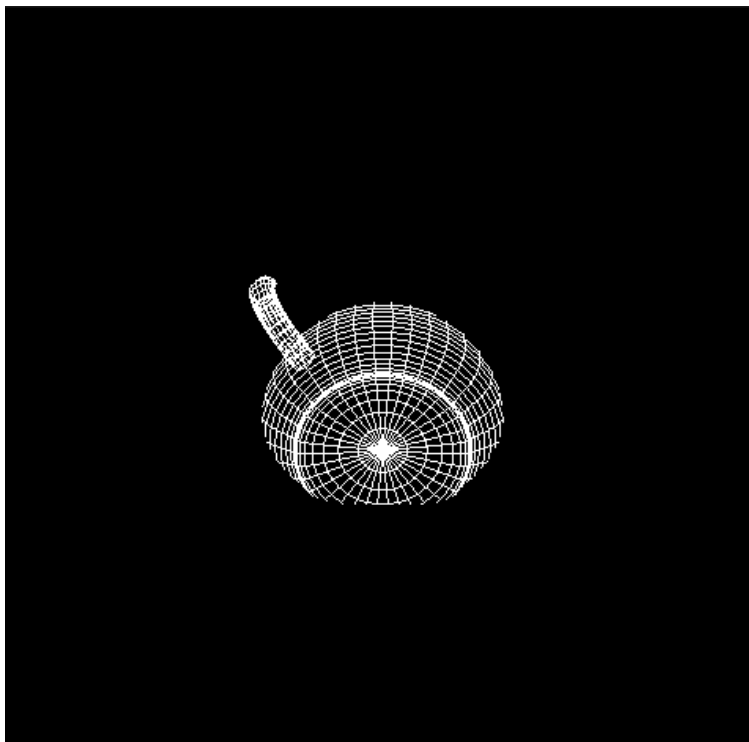


FIGURE 3 – rendu pour la valeur 2.8 de znear

Positionnement : Inverser le vecteur "Up" de la caméra (mettre -1 au lieu de 1) fait basculer la scène à l'envers. Nous avons utilisé les configurations suivantes pour obtenir les images voulues en vérifiant pour chaque configuration que le vecteur up n'est pas colinéaires au vecteur lookatpoint :

- **Vue 3/4 :** `gluLookAt(0.0, 0.0, 2.2, 0.0, 0.0, -1.0, 1.0, 1.0, 0.0)`
- **Vue de dessus :** `gluLookAt(0, 2.0, 0, 0.0, -1.0, 0, 1, 0.0, 0.0)`

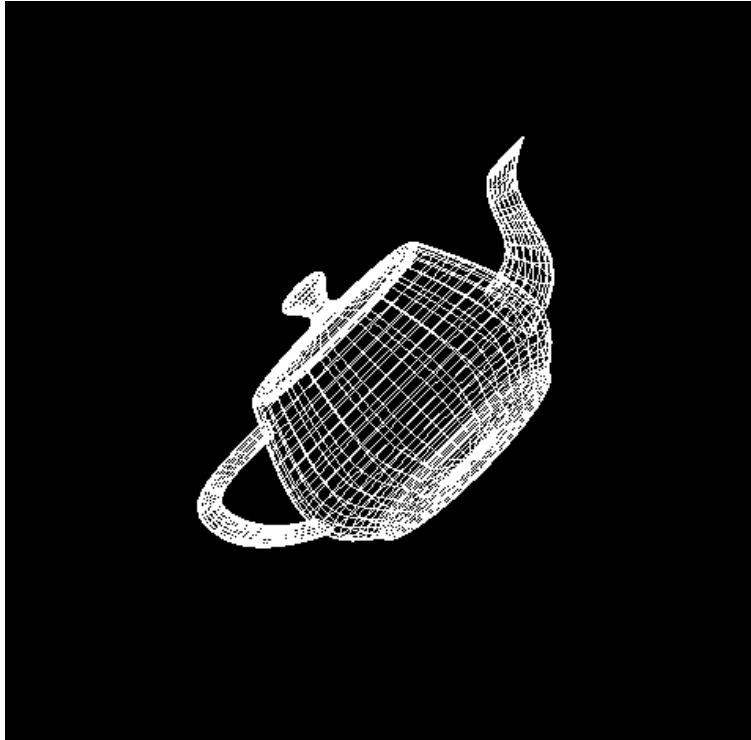


FIGURE 4 – Vue de trois-quarts inclinée

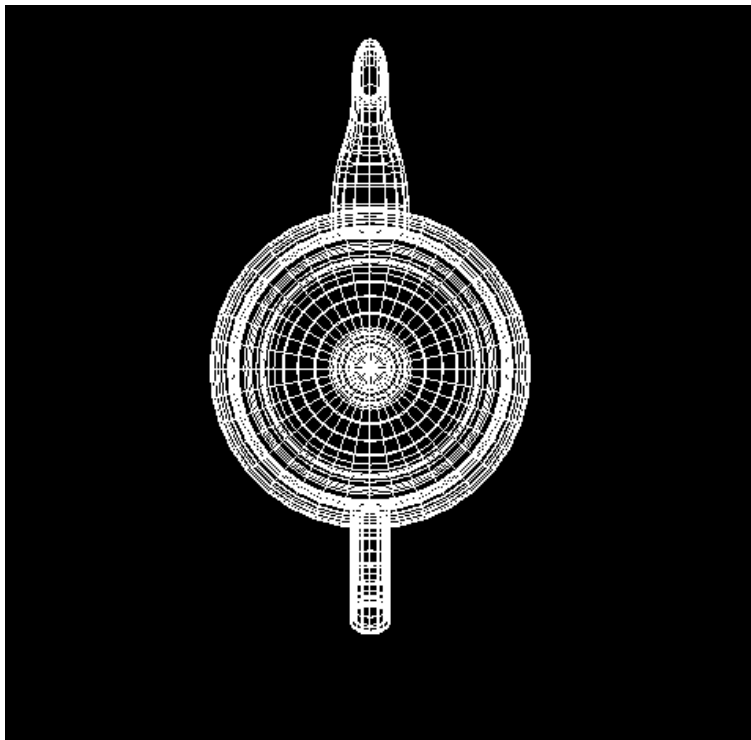


FIGURE 5 – Vue de dessus (Axe Y)

4 Exercice 3 : La Pile de Matrices

Gestion du Push/Pop : L'oubli du deuxième `glPopMatrix()` fait que les transformations s'accumulent à chaque fois que l'écran se rafraîchit. Résultat : les objets dérivent, se multiplient ou finissent par sortir de l'écran.

Emplacement de `gluLookAt` : Si on place la caméra après un `glPushMatrix`, son effet est supprimé dès le `glPopMatrix` suivant. La caméra doit être définie avant de dessiner les objets.

Réinitialisation : Utiliser `glLoadIdentity()` dans la boucle d'affichage efface les réglages de la caméra faits dans le `main`. On se retrouve alors "collé" à l'objet ou à l'origine du repère, sans pouvoir voir la scène correctement.

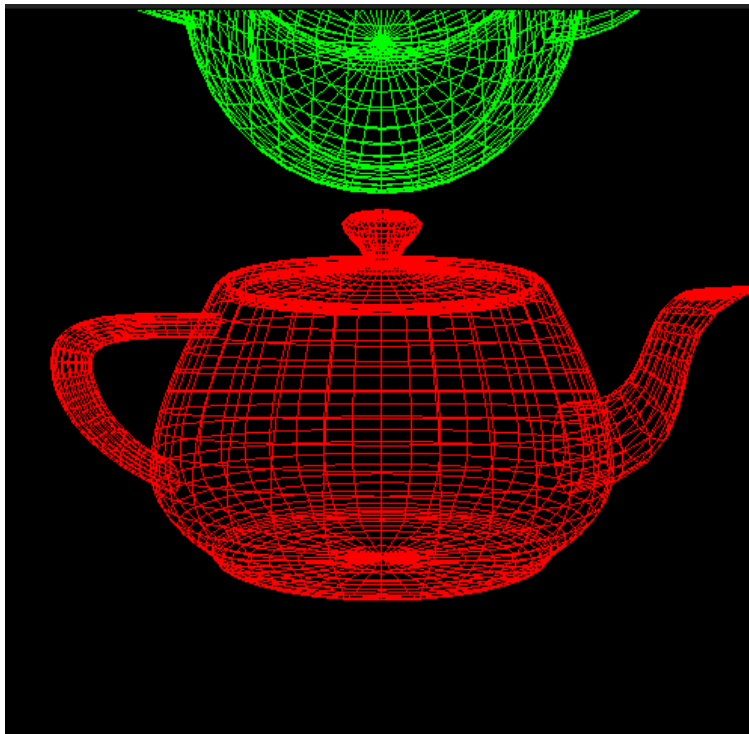


FIGURE 6 – Résultats des questions 2 et 3

5 Exercice 4 : Navigateur Interactif

L'objectif de cet exercice est de créer une caméra mobile pour se déplacer librement dans la scène 3D. Contrairement aux exercices précédents, la position et l'orientation de la vue ne sont plus fixes mais dépendent de variables mises à jour par l'utilisateur.

5.1 Logique de la vue et calcul de direction

Pour que la caméra puisse regarder dans toutes les directions, nous utilisons deux angles de rotation : `rotX` (vertical) et `rotY` (horizontal). Avant chaque rendu, ces angles sont convertis en radians pour calculer un vecteur de direction ($dirX, dirY, dirZ$) :

- $dirX = \sin(radY) \times \cos(radX)$
- $dirY = \sin(radX)$
- $dirZ = -\cos(radY) \times \cos(radX)$

Ce vecteur est ensuite ajouté à la position actuelle de la caméra ($camX, camY, camZ$) dans la fonction `gluLookAt`. Cela permet à l'observateur de toujours regarder "devant lui" selon l'orientation choisie.

5.2 Contrôles et Interactivité

Le déplacement est géré par deux types de fonctions de rappel (*callbacks*) :

- **Clavier classique (keyboard)** : Les touches **W**, **A**, **S**, **D** permettent de déplacer la position de la caméra sur les axes X et Z (avant/arrière et gauche/droite), tandis que **Q** et **Z** gèrent la hauteur (axe Y).
- **Touches spéciales (specialKeys)** : Les flèches directionnelles permettent de modifier les angles de rotation `rotX` et `rotY` pour orienter le regard.

```
1 float radX = rotX * PI / 180.0f;
2 float radY = rotY * PI / 180.0f;
3
4 float dirX = sinf(radY) * cosf(radX);
5 float dirY = sinf(radX);
6 float dirZ = -cosf(radY) * cosf(radX);
7
8 gluLookAt(camX, camY, camZ,
9           camX + dirX, camY + dirY, camZ + dirZ,
10          0.0, 1.0, 0.0);
```

Listing 1 – Calcul de la direction dans la fonction `display`

Chaque pression de touche appelle `glutPostRedisplay()`, ce qui force OpenGL à redessiner la scène avec les nouvelles coordonnées, créant ainsi un mouvement fluide.

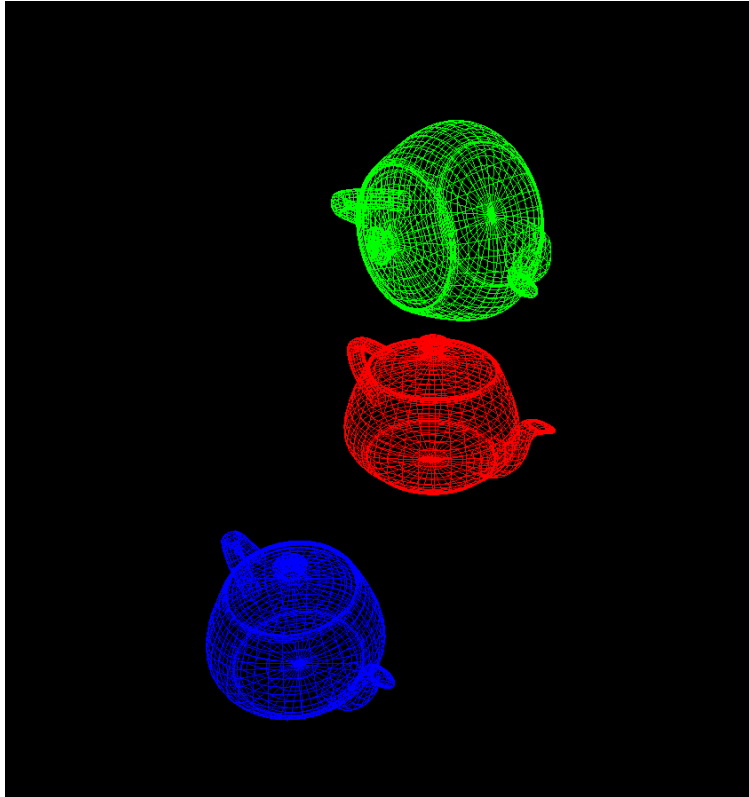


FIGURE 7 – Exemple de rendu de navigator

6 Conclusion

Nous avons appris à placer une caméra et à manipuler des objets 3D en respectant l'ordre des transformations dans la pile OpenGL.